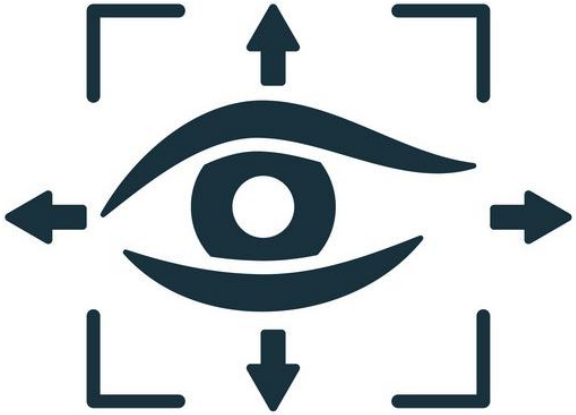
Eye Tracking

Dr. Salwa Osama



What exactly is eye tracking?

- Eye tracking is a technology that allows us to detect where someone is looking in real-time.
- Whether it’s determining which part of a screen a person is focusing on or tracking the movement of eyes as they scan a page
- Uses of eye tracking:
- **Human-Computer Interaction (HCI):** controlling a computer just by looking at it.
- **Medical Diagnostics:** analyzing eye movements to detect neurological conditions.
- **Gaming:** creating more immersive experiences by tracking where players are looking.

OpenCV library

- OpenCV = **Open Source Computer Vision Library**.
- Free and open-source.
- Used for **image processing** and **computer vision** tasks.
- Works with many languages: **Python, C++, Java**.
- Supports multiple platforms: **Windows, Linux, MacOS, iOS, Android**.

Why use OpenCV

- Designed for **real-time applications**.
- Has **powerful tools** for face recognition, object detection, image filtering.
- Supports **machine learning** and AI tasks.
- Works with **cameras and videos** easily.
- Used in research, robotics, games, and medical applications.

You can install it easily using pip

```
pip install opencv-python
```

Video Capture

This library is used for image and video processing.

Opens the **webcam**. **0** means the default (first) camera on your device.
If you have more cameras, you can use 1, 2, etc.

```
import cv2
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    cv2.imshow('Video Feed', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

cap.read() returns two things:

ret: True/False → tells if the frame was captured correctly.

frame: the actual image (video frame) from the webcam.

The loop keeps running to continuously capture frames.

Video Capture

Waits **1 millisecond** to check if a key was pressed.
If the user presses the **q key**, the program stops (break).

```
import cv2
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    cv2.imshow('Video Feed', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

cap.release(): releases the webcam.
cv2.destroyAllWindows(): closes all OpenCV windows.

Video Capture

Makes the video appear in black and white.

```
import cv2
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    cv2.rectangle(frame, (50, 50), (200, 200), (0, 255, 0), 8)
    cv2.putText(frame, "Hello OpenCV", (50, 40), cv2.FONT_HERSHEY_SIMPLEX,
            1, (255, 0, 0), 2)

    cv2.imshow('Video Feed', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

- top-left corner of the rectangle (x=50, y=50).
- OpenCV uses a coordinate system where (0,0) is the top-left of the image, x goes right, y goes down.
- **(200, 200)** — bottom-right corner of the rectangle (x=200, y=200).
- **(0, 255, 0)** — rectangle color in **BGR** order (Blue, Green, Red). (0,255,0) = bright **green**.
- **3** — line thickness in pixels. If you put -1 instead, the rectangle is **filled**.

Video Capture

```python
import cv2
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    cv2.rectangle(frame, (50, 50), (200, 200), (0, 255, 0), 8)
    cv2.putText(frame, "Hello OpenCV", (50, 40), cv2.FONT_HERSHEY_SIMPLEX,
                1, (255, 0, 0), 2)

    cv2.imshow('Video Feed', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

- **(50, 40)** — the **bottom-left** corner of the text baseline (x=50, y=40).
- Note: text origin is the lower left of the text, not the top-left.
- **cv2.FONT_HERSHEY_SIMPLEX** — font family (a built-in OpenCV font).
- **1** — fontScale (size multiplier). 1 is the base size, 2 is larger, 0.5 is smaller, etc.
- **(255, 0, 0)** — text color in **BGR** → this is **blue**.
- **2** — text thickness in pixels.

Simple Explanation of Eye Tracking

- Eye tracking is the process of determining where someone is looking. But there's more to it than just that. Eye tracking involves a few key components:
  - **Detecting the Eye:** First, we need to locate the eyes in an image or video frame. This usually involves facial detection followed by zooming in on the eye regions.
  - **Identifying the Pupil:** Once the eyes are detected, the next step is to find the pupil, which is typically the darkest part of the eye. This is crucial because the position of the pupil helps us determine the direction.
  - **Tracking Movement:** Finally, we track the movement of the eyes or pupils over time. By continuously monitoring the eye positions, we can infer where someone is looking or how their gaze shifts.

Types of Eye Tracking:

Eye tracking isn't one-size-fits-all. There are different methods depending on the level of accuracy you need and the resources at your disposal. Here are the two main types:

Feature-based Eye Tracking:

This method relies on specific features of the eye, like the pupil or iris, to track movement. It's usually faster but might not be as accurate in challenging conditions, like low light.

Example: You can use simple thresholding and contour detection to isolate and track the pupil. Here's how you might approach it:

Appearance-based Eye Tracking:

This method uses the overall appearance of the eye region, often employing machine learning or deep learning techniques. It's more robust but computationally expensive.

Example: You might train a convolutional neural network (CNN) to predict gaze direction based on the eye region's appearance.

Feature-based Eye Tracking

```
ret, thresh = cv2.threshold(eye_gray, 70, 255, cv2.THRESH_BINARY_INV)
contours, _ = cv2.findContours(thresh, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
contours = sorted(contours, key=lambda x: cv2.contourArea(x), reverse=True)

if contours:
    (cx, cy, cw, ch) = cv2.boundingRect(contours[0])
    cv2.rectangle(eye_color, (cx, cy), (cx+cw, cy+ch), (0, 0, 255), 2)
```

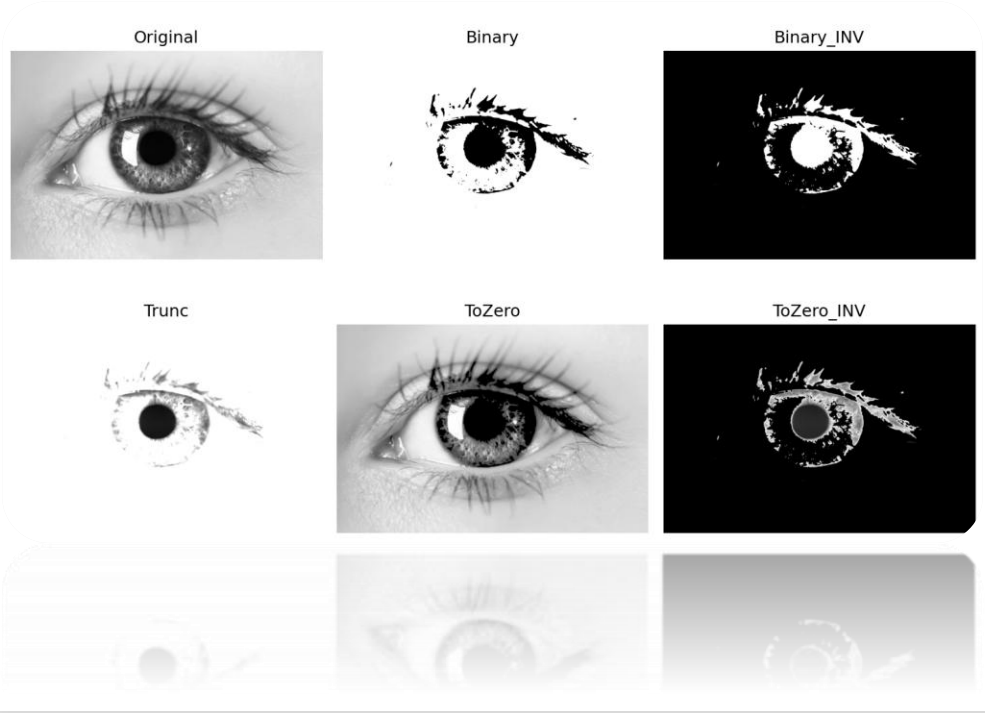
Thresholding

```
img = cv2.imread('AI COdes/HCI Course/Basics of HCI/eye.jpg',0)  #0 means v2.IMREAD_GRAYSCALE, 1 is colored
```

- **Original:** The original image in grayscale.
- **Binary:** Any pixel with a value greater than 70 becomes 255 (white), and the rest become 0 (black).
- **Binary_INV:** The opposite of Binary → any pixel greater than 70 becomes 0 (black), and the rest become 255 (white).
- **Trunc:** Any pixel value greater than 70 is set to 70, while the rest stay unchanged.
- **ToZero:** Any pixel less than 70 becomes 0 (black), and the rest stay unchanged.
- **ToZero_INV:** The opposite of ToZero → any pixel greater than 70 becomes 0, and the rest stay unchanged.

```
_, th_binary = cv2.threshold(img, 70, 255, cv2.THRESH_BINARY)
_, th_binary_inv = cv2.threshold(img, 70, 255, cv2.THRESH_BINARY_INV)
_, th_trunc = cv2.threshold(img, 70, 255, cv2.THRESH_TRUNC)
_, th_tozero = cv2.threshold(img, 70, 255, cv2.THRESH_TOZERO)
_, th_tozero_inv = cv2.threshold(img, 70, 255, cv2.THRESH_TOZERO_INV)
```

Thresholding



Original: The original image in grayscale.

Binary: Any pixel with a value greater than 70 becomes 255 (white), and the rest become 0 (black).

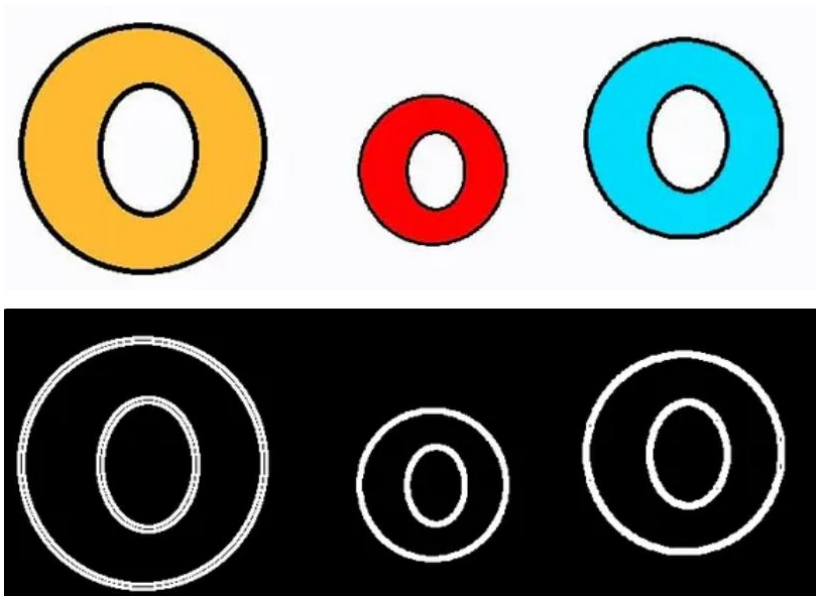
Binary_INV: The opposite of Binary → any pixel greater than 70 becomes 0 (black), and the rest become 255 (white).

Trunc: Any pixel value greater than 70 is set to 70, while the rest stay unchanged.

ToZero: Any pixel less than 70 becomes 0 (black), and the rest stay unchanged.

ToZero_INV: The opposite of ToZero → any pixel greater than 70 becomes 0, and the rest stay unchanged.

Contour



- Contours are useful in many applications such as object detection, shape analysis, and image segmentation. By detecting contours, you can:
- Identify the boundaries of objects within an image.
- Analyze shapes to determine their properties (e.g., area, perimeter).
- Segment images by separating objects from the background.

Steps to find Contour

- **Convert to Grayscale**: Convert the image to a grayscale image, which simplifies the processing.
- **Apply Thresholding**: Convert the grayscale image to a binary image by applying a threshold.
- **Detect Contours**: Use the `find Contours` function to detect contours in the binary image.

Steps to find Contour

Convert to Grayscale

Apply Thresholding

```
import numpy as np
import matplotlib.pylab as plt
image = cv2.imread("AI COdes/HCI Course/Basics of HCI/eye.jpg", cv2.IMREAD_GRAYSCALE)
__, thresh = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY_INV)
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
contour_image = np.zeros_like(image, dtype=np.uint8)
cv2.drawContours(contour_image, contours, -1, (255, 255, 255), 2)
cv2.imwrite('contour.jpg', contour_image)
plt.imshow(contour_image, cmap="gray")
plt.show()
```

Detect Contours

Contour detection

cv2.RETR_EXTERNAL → the retrieval mode, meaning we only keep the **outermost contours** (ignore inner nested ones).

```
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

contours → a list of all the detected contours. Each contour is stored as a list of points (x, y).
_ → the hierarchy (relationship between contours). We don't need it here, so we use _

cv2.CHAIN_APPROX_SIMPLE → contour approximation method, which compresses horizontal/vertical/diagonal segments and saves only the **endpoints**, reducing memory usage.

1. Face Detection

Load the **Haar Cascade classifier** for face detection. This file contains the trained model to detect human faces.

```python
import cv2
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 0, 0), 2)

    cv2.imshow('Face Detection', frame)
```

1.3 → **Scale Factor**: specifies how much the image size is reduced at each image scale.

- Smaller value (1.1) = more accuracy but slower.
- Larger value (1.3 or 1.4) = faster but may miss some faces.

5 → **minNeighbors**: defines how many neighbor rectangles need to agree to keep a detection.

- Higher = fewer false positives but stricter detection.
- Lower = more faces detected but higher chance of false positives.

2. Eye Detection

```
faces = face_cascade.detectMultiSc

for (x, y, w, h) in faces:
    cv2.rectangle(frame, (x, y), (x
    roi_gray = gray[y:y+h, x:x+w]
    roi_color = frame[y:y+h, x:x+w]

    eyes = eye_cascade.detectMultiScale(roi_gray)
    for (ex, ey, ew, eh) in eyes:
        cv2.rectangle(roi_color, (ex, ey), (ex+ew, ey+eh), (0, 255, 0), 2)
```

Inside the face region, use the **eye Haar Cascade** to detect eyes. Each detected eye is returned as (ex, ey, ew, eh)

3. pupil Detection

Apply **thresholding** to isolate the pupil: Dark pixels (pupil) → white. Bright pixels (skin/eye) → black.

```
eyes = eye_cascade.detectMultiScale(roi_gray)
for (ex, ey, ew, eh) in eyes:
    eye_gray = roi_gray[ey:ey+eh, ex:ex+ew]
    eye_color = roi_color[ey:ey+eh, ex:ex+ew]

    _, thresh = cv2.threshold(eye_gray, 70, 255, cv2.THRESH_BINARY_INV)

    contours, _ = cv2.findContours(thresh, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
    contours = sorted(contours, key=lambda x: cv2.contourArea(x), reverse=True)

    if contours:
        (cx, cy), radius = cv2.minEnclosingCircle(contours[0])
        cv2.circle(eye_color, (int(cx), int(cy)), int(radius), (255, 0, 0), 2)
```

Find **contours (boundaries)** of white regions. Sort contours by area.The largest contour usually = pupil.

Compute the **minimum enclosing circle** around it (pupil).Draw a **blue circle** to show the pupil position.

Thanks